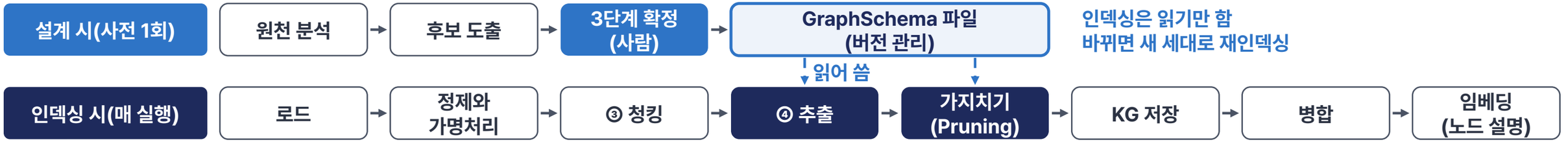
1. 스키마 도출: 노드 타입과 관계는 추출 전에 먼저 정함

스키마는 추출 결과가 아니라 추출기에 주는 허용 목록 — 먼저 정하지 않으면 같은 개념이 여러 라벨로 쪼개짐



스키마 없이 추출하면 (실측)

로컬 gemma-4 12B로 청크 2개를 스키마 없이 추출 → 라벨 11종

Brand · CardProduct = 카드 하나가 라벨 2개로

Fee · AnnualFee = 요금도 라벨 2개로

Channel · Benefit · Guardian · Grade · Date

Store · PerformanceCondition

GraphRAG 도구 기본 노드 타입/관계가 없거나 너무 일반적임

- Neo4j: 스키마를 안 주면 매번 입력 글에서 자동 추출(사람 검토 없음)
- MS GraphRAG 기본 타입: organization, person, geo, event
- LangChain+Neo4j: 허용 목록이 비면 아무 타입이나 만듦

스키마를 주면 (Native GraphSchema)

```
schema = GraphSchema.model_validate({
    "node_types": ["Card", "Benefit", "Fee"],
    "relationship_types": ["HAS_BENEFIT", "HAS_FEE"],
    "patterns": [("Card", "HAS_BENEFIT", "Benefit"),
                 ("Card", "HAS_FEE", "Fee")]})
graph = await extractor.run(chunks=chunks, schema=schema)
result = await GraphPruning().run(graph, schema)
```

① 추출 안내
허용 목록이 추출 프롬프트에 들어가 LLM이 그 안에서 고름

② 가지치기
목록 밖 노드와 관계를 지우고 지운 개수를 통계로 남김

실행 확인: 노드 타입을 주면 목록 밖 타입은 기본 불허

결론: 스키마는 인덱싱 전 설계 단계에서 확정해 파일로 고정, 인덱싱은 읽어 쓰기만 함.
정형 테이블은 변환 규칙으로, 문서는 표본 → 후보 → 사람 확정으로 스키마 설계

무단전재 및 배포 금지

1. 스키마 도출: 스키마에 정의하는 7가지

노드 타입과 관계, 각각의 속성, 연결 패턴이 뼈대 — 제약과 허용 범위로 무엇을 지킬지 정함

뼈대 ① ~ ⑤ 왼쪽부터 정의할 것, GraphSchema 칸, 카드 예

①	**노드 타입**	node_types[].label	**Card, Benefit**
②	**노드 속성**	node_types[] .properties	**Card {name, annual_fee}**
③	**관계**	relationship_types[] .label	**HAS_BENEFIT**
④	**관계 속성**	relationship_types[] .properties	**HAS_BENEFIT {valid_until}**
⑤	**연결 패턴**	patterns (출발, 관계, 도착)	**(Card, HAS_BENEFIT, Benefit)**

지킬 규칙 ⑥ ⑦

⑥	**제약**	constraints UNIQUENESS, EXISTENCE, KEY	**Card.name = KEY**
⑦	**허용 범위**	additional_node_types, additional_relationship_types, additional_patterns	**①~⑤에 지정된 목록 밖을 허용할지 여부**



7칸을 채운 Native GraphSchema — 실행 확인: name 없는 Card는 가지치기가 지움

```
schema = GraphSchema.model_validate({
  "node_types": [                          # ① ②
    {"label": "Card", "properties": [
       {"name": "name", "type": "STRING"},
       {"name": "annual_fee", "type": "INTEGER"}]},
    {"label": "Benefit", "properties": [...]}],
  "relationship_types": [{"label": "HAS_BENEFIT",   # ③ ④
     "properties": [{"name": "valid_until", "type": "DATE"}]}],
  "patterns": [("Card", "HAS_BENEFIT", "Benefit")],   # ⑤
  "constraints": [{"type": "KEY", "node_type": "Card",  # ⑥
                   "property_names": ["name"]}],
  "additional_node_types": False})              # ⑦
```

※ 최소한 = ①, ③, ⑤ + ⑥ 고유키. 속성은 질문에 쓰는 것만 정의하고, 타입마다 description을 적으면 LLM이 고르는 기준이 됨
※ 제약: 지정한 속성에 대해 UNIQUENESS는 고유 강제, EXISTENCE는 존재 강제, KEY는 고유 + 존재를 강제함. UNIQUENESS는 지정한 속성이 비어 있어도 허용하나 KEY는 불허함

무단전재 및 배포 금지

1. 스키마 도출: 정형 테이블 → 노드 타입과 관계: 변환 규칙

모델 없이 규칙으로 바꿈 — 단, FK를 세기 전에 '그 표 자체가 조회 대상인가'부터 물음

테이블 1개

Q1. 그 표 자체가 조회 대상인 엔티티인가?

예

① 노드 타입
열 = 노드 속성
FK = 관계
FK 열은 바꾼 뒤 지움

FK가 0개 또는 1개인
보통 테이블은 대부분 여기로 옴

아니오

Q2. FK 몇 개?

② 조인 테이블: 2개이고 업무 열 없음
관계 1개로 흡수

③ 조인 테이블: 2개 + 업무 속성
관계 + 관계 속성
(조회하거나 집계하면 노드)

④ 3개 이상
노드 타입(중간 노드)

왜 Q1이 먼저? FK 수만 보면 이체 기록(FK 2개)이 관계로 흡수돼 이체 자체를 찾을 수 없음 (예 ①)

① 이체(출금계좌FK, 입금계좌FK, 금액, 일시) → **:계좌** —:출금→ **:이체** —:입금→ **:계좌**
조회 대상 → 노드 타입(FK가 2개여도)

② 사용자, 역할, 사용자역할(사용자FK, 역할FK) → **:사용자** —:맡음→ **:역할**
업무 열 없음 → 관계로 흡수(사용자역할 노드 없음)

③ 주문상품(주문FK, 상품FK, 수량, 단가) → **:주문** —:포함 {수량, 단가}→ **:상품**
업무 속성 → 관계 속성(판매 집계가 잦으면 노드)

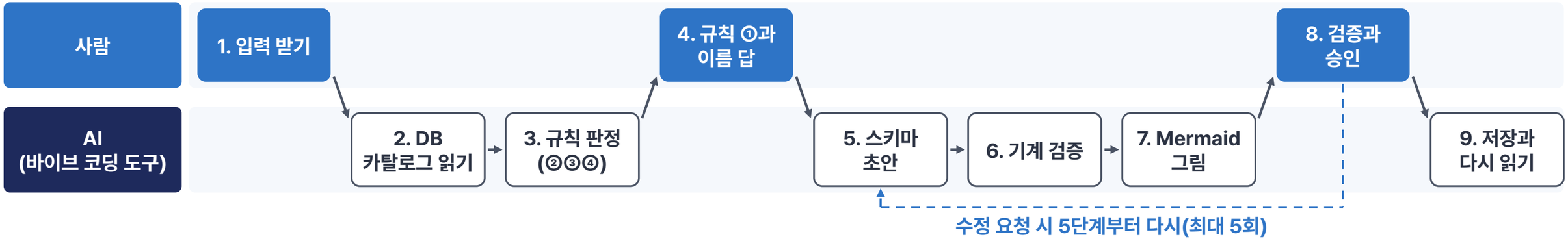
④ 수강신청(학생FK, 과목FK, 학기FK) → **:학생** —:신청→ **:수강신청** —:대상→ **:과목**
FK 3개 → 중간 노드(학기도 같은 방식), 홉 +1

Neo4j 공식 변환 규칙과 같음: 테이블 → 라벨 , FK → 관계 , 조인 테이블 → 관계 또는 관계 + 속성, 셋 이상 → 중간 노드
이름에 공백 없음(Neo4j 권고: 라벨은 PascalCase, 관계는 대문자_밑줄). 정형에서 나온 타입은 LLM 추출 비용이 들지 않음

무단전재 및 배포 금지

1. 스키마 도출: 프롬프트로 정형 테이블의 스키마 도출

변환 규칙을 프롬프트에 담아 바이브 코딩 도구에 맡김 — AI는 읽고 판정하고 그리며, 사람은 세 번 답하고 승인함



사람이 답하는 곳 3번

1 출력 폴더와 DB 접속 정보
비밀번호는 .env에 직접 등록

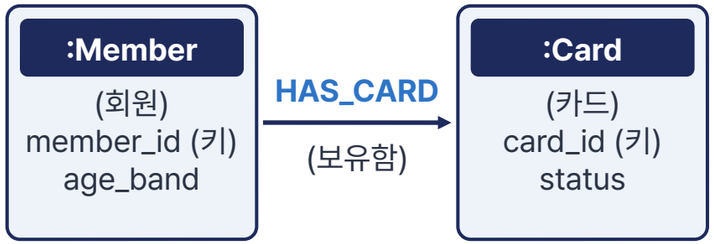
4 규칙 ① 후보(FK 2개 + 업무 열)를 노드 타입
또는 관계 + 관계 속성으로 정함
뺄 테이블, 이름 후보 승인

8 Mermaid 그림과 판정표를 보고 승인
(승인자와 일시를 기록)

AI가 지키는 규칙

- FK 개수는 열 수가 아니라 제약 수로 셈
- DB는 읽기 전용, 비밀번호는 .env에만 둠
- 라벨은 영문, 한글 이름은 description에 넣음
- 그림은 Mermaid만 씀
- 규칙 ① 판정과 이름은 추측해 확정하지 않음
- 후보 표시가 남으면 스키마 파일을 만들지 않음

승인 화면 예 (Mermaid 그림)



영문 라벨 아래에 한글 이름을 함께 적어
검토하는 사람이 바로 읽게 함

결과: graph_schema.json(승인본)과 approval.json(승인자, 승인 일시, 입력 파일 SHA-256). 인덱싱은 이 파일을 읽기만 함
GraphSchema 검증, 저장, 다시 읽기는 Neo4j 서버 없이 됨(실행 확인)

※ DB 카탈로그를 읽을 스키마는 원본 테이블이 있는 스키마로 지정하고, 서비스용 뷰(View)가 정의된 스키마(예: app)는 쓰지 않음

무단전재 및 배포 금지

1. 스키마 도출: 소스 문서 → 노드 타입과 관계: 권고 절차(Neo4j)

LLM은 넓게 후보만 뽑고, 사람이 역량 질문 기준으로 뼈대를 확정한 뒤, 뼈대로 실제 추출해 검증하고 GraphSchema로 고정함

① 역량 질문 정하기 사람 → **② 넓게 뽑기 LLM** → **③ 뼈대 확정 사람** → **④ 채우기 시험 LLM + 사람** → **⑤ 고정 코드**

④ → ③ : 설명 고쳐 다시

- 문서 종류마다 답해야 할 질문 5개 안팎
- 타입 이름은 쓰지 않고 질문으로만 씀
- 질문이 범위이자 나중의 합격 기준

- 층별 표본: 시작 10건, 상한 30건(비율이 아니라 건수)
- SchemaFromTextExtractor에 질문을 examples로 넣음
- 앞서 나온 이름을 다시 쓰라는 안내는 넣지 않음

- 합치기, 빼기, 보태기
- 빈도가 아니라 질문 기준(드문 이름도 살림)
- 타입마다 description을 좁게 (무엇이 아닌지까지)
- 제약은 사람이 정함

- 뼈대를 schema로 주고 추출한 뒤 GraphPruning으로 정리
- 정답표나 질문의 답이 채워지는지 셈
- 잘못 뽑기가 많으면 설명을 고쳐 ③으로

- GraphSchema.save(), additional_* = false
- 인덱싱은 이 파일을 읽기만 함
- 바뀌면 새 세대로 다시 인덱싱

주의

! 앞서 나온 이름을 다시 쓰라는 안내는 이름을 모으지만 새 개념을 막음 → ④에서만

! 대화 속 행위나 조항 규칙 같은 구조는 LLM이 잘 못 뽑음 → 사람이 보탬

! 여러 문서에 걸친 관계(카드 비교 등)는 청크 단위 추출로 나오지 않음

! 멈춤: 2건 이상 나온 새 이름이 연속 3건 없으면 그 층은 멈춤

출처: Neo4j GraphRAG Python 사용자 안내, Neo4j LLM Graph Builder 기능, Noy와 McGuinness, Ontology 101(역량 질문), Guest 외(2020) 포화 기준

무단전재 및 배포 금지

1. 스키마 도출: 사례 ① 상담 이력: 넓게 뽑기에서 뼈대 확정까지

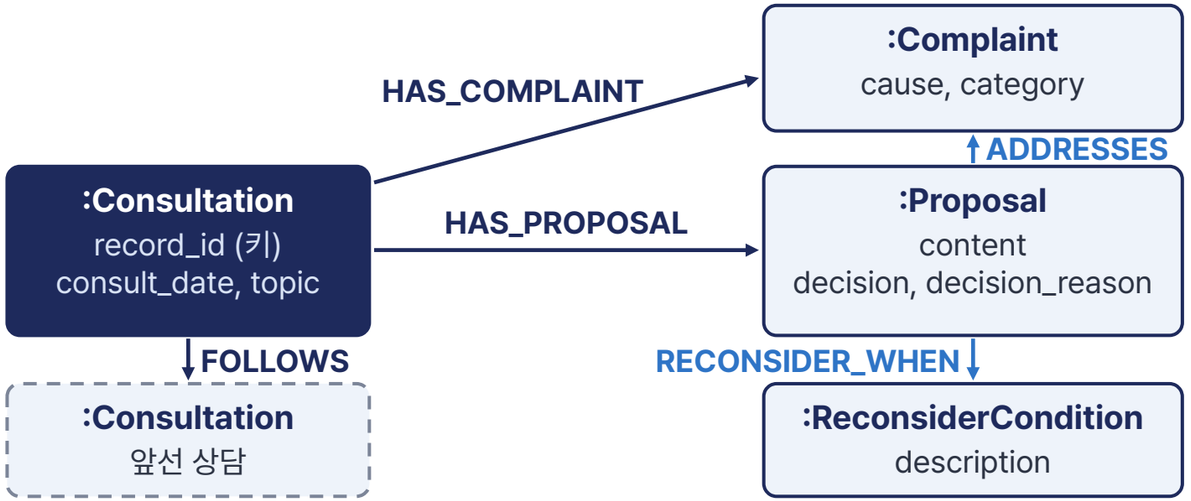
같은 상담 30건에 방법만 바꿔 보니, 역량 질문만 넣었을 때 비로소 불만, 제안, 응답, 재검토 조건이 후보로 나옴

같은 상담 30건, 방법만 바꿔 봄

방법	노드 타입 이름 수	업무 개념 4개 중
1건씩 따로 뽑기	45	0 / 4
이름 다시 쓰기 (앞서 나온 이름을 examples로)	14	0 / 4
원문 보강 + 이름 다시 쓰기	38	0 / 4
원문 보강 + 역량 질문 + 이름 다시 쓰기	10	1 / 4
원문 보강 + 역량 질문만	**53**	**4 / 4**

업무 개념: 불만 원인, 제안, 수용과 거절, 재검토 조건
원문 보강: 상담 원문에 그 문장을 넣은 판(v3)
역량 질문만 넣으면 네 개념이 각각 30건 중 1 ~ 5건뿐이라 빈도로 거르면 버려짐

확정한 뼈대(역량 질문 기준으로 사람이 정함)



네이비: LLM 후보를 합침, 파랑: 사람이 보탬(관계 2개와 decision 속성)

합침: Issue와 Complaint → Complaint, Proposal과 SUGGESTS → Proposal, Conversation과 Session → Consultation
뺌: Transaction(RDB 집계), Customer의 이름과 이메일(가명처리 대상), Request와 Menu(질문에 안 쓰임)

결론: 드문 핵심 개념은 역량 질문만으로 넓게 뽑아야 보이고, 무엇을 살릴지는 빈도가 아니라 질문을 기준으로 사람이 정함

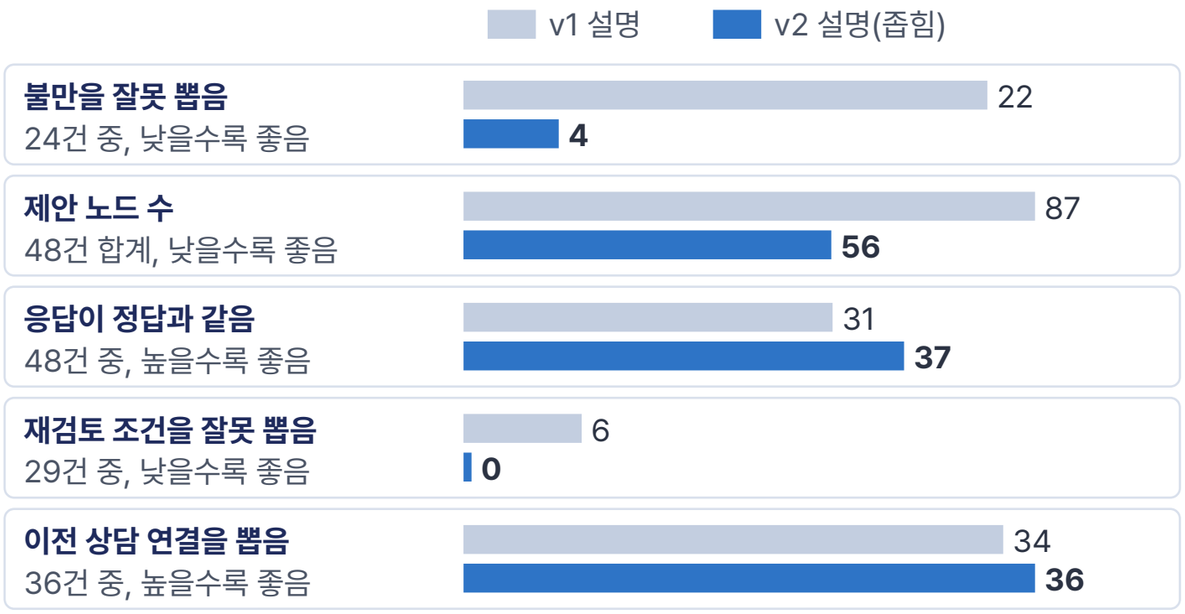
실측: 2026-10-18, LLM Groq openai/gpt-oss-120b, neo4j-graphrag 1.22.0 SchemaFromTextExtractor, 합성 상담 이력 30건(같은 순서)

무단전재 및 배포 금지

1. 스키마 도출: 사례 ② 뼈대 채우기 시험과 문서 종류별 차이

뼈대의 이름보다 설명(description)이 추출 품질을 좌우함 — 채우기 시험으로 설명을 고쳐 가며 확정

뼈대 채우기 시험(상담 48건, 정답표와 비교)



Complaint 설명 v1: 고객이 말한 불만과 그 원인
v2: 불만이나 불편을 분명히 말한 경우만. 문의 목적과 개인 사정(이사, 지출 감소)은 아님

문서 종류에 따라 비중이 다름

혜택 안내

이름이 빨리 모임(핵심 6개). 역량 질문을 넣자 제외 대상 Exclusion이 3건에서 25건으로

→ 넓게 뽑은 후보가 거의 그대로 뼈대, 사람은 합치기 위주

상담 이력

대화 속 행위(불만, 제안, 응답)는 30건 중 1 ~ 5건만 나옴

→ 사람이 보태고, 채우기 시험으로 설명을 다듬음

약관

청크마다 조항이 달라 1건에서만 나온 이름이 76%. 기한과 의무 같은 규칙 구조는 거의 안 나옴

→ 조항 구조 뼈대를 사람이 먼저 그림

표본 수: 이름 다시 쓰기 시험에서 혜택 층은 10 ~ 14건, 상담은 13 ~ 20건에서 멈춤(상한 30건 전)

결론: 절차 ①~⑤는 같음. 문서 성격에 따라 LLM 후보를 그대로 쓰는 비중과 사람이 설계하는 비중이 달라짐

실측: 2026-10-18, Groq openai/gpt-oss-120b, neo4j-graphrag 1.22.0 LLMEntityRelationExtractor와 GraphPruning, 정답: 강사용 상담근거 대조표(v3)

무단전재 및 배포 금지